

# Archivo 5 — Material Didáctico

## Organización de este material

Este documento desarrolla la teoría de **todos** los conceptos presentados en la Guía de Estudio (Archivo 4), organizados en las mismas seis unidades de dificultad creciente y orientados a la resolución concreta que el proyecto zz-js01 realiza. Cada apartado sigue el mismo esquema:

1. **Teoría:** la definición del concepto, construida desde cero.
2. **Ejemplo genérico:** un ejemplo breve y autónomo, que no pertenece al proyecto.
3. **Fragmento real:** un extracto literal de un archivo del proyecto, con su ruta.
4. **Conexión con el proyecto:** la frase que une el concepto con la resolución.

Los fragmentos reales son breves y conviven con la teoría que ilustran; nunca se presenta un archivo completo. Se recomienda tener abierto el archivo citado para observar el fragmento en su contexto.

## Unidad 1 — Fundamentos del lenguaje (Nivel 1)

### 1.1 Variables: `const` y `let`

**Teoría.** Un programa necesita guardar valores con un nombre para poder usarlos más adelante: eso es una variable. JavaScript ofrece dos palabras principales para crearlas. `const` crea una variable cuyo nombre queda fijado: no puede reasignarse a otro valor. `let` crea una variable cuyo valor puede cambiar durante la ejecución. La regla práctica — que el proyecto sigue de forma sistemática — es usar `const` por defecto y reservar `let` para los valores que efectivamente cambian.

```
// Ejemplo genérico (no pertenece al proyecto)
const precioFijo = 10; // const: el nombre no se reasigna
let total = 0;          // let: el valor puede cambiar
total = total + precioFijo;
```

**Fragmento real** — `public/index.html` . El estado que cambia entre "crear" y "editar" se declara con `let` ; las referencias fijas de la página, con `const` :

```
// Estado del formulario. idEnEdicion: null = creando un producto
// nuevo; un número = editando ese id. Esta única variable decide
// si el submit manda POST (crear) o PUT (actualizar).
let idEnEdicion = null;
```

**Conexión con el proyecto:** una única variable `let` — `idEnEdicion` — gobierna la diferencia entera entre crear un producto y editarlo.

## 1.2 Tipos de datos primitivos

**Teoría.** Cada valor en JavaScript tiene un tipo. Los tipos primitivos básicos son: `string` (texto, entre comillas), `number` (números enteros y decimales), `boolean` (`true` o `false`), `undefined` (a un nombre aún no se le asignó valor) y `null` (vacío intencional). El operador `typeof` devuelve el tipo de un valor como texto, y resulta indispensable para validar datos antes de operar con ellos, porque un texto y un número no se comportan igual.

```
// Ejemplo genérico (no pertenece al proyecto)
typeof "hola"      // "string"
typeof 10.5        // "number"
typeof true        // "boolean"
typeof undefined   // "undefined"
```

**Fragmento real** — `src/validators/productos.js`. Antes de medir el largo del nombre, se verifica que sea realmente texto:

```
if (typeof datos.nombre !== "string" || datos.nombre.trim() === "") {
  errores.push("El nombre es obligatorio y debe ser texto.");
}
```

**Conexión con el proyecto:** la validación del nombre empieza por el tipo — si no llega un `string`, medir su largo no tendría sentido.

## 1.3 Funciones: declaración y funciones flecha

**Teoría.** Una función es un bloque de código con nombre que se ejecuta cuando se la llama; puede recibir valores (parámetros) y devolver un resultado con `return`. JavaScript ofrece dos sintaxis principales. La declaración clásica escribe `function nombre(parámetros) { ... }`. La **función flecha** (arrow function) escribe los parámetros, el símbolo `=>` y el cuerpo: `(p) => p.id === id`. Las funciones flecha predominan cuando la función se pasa como valor a otra — por ejemplo, como manejador de un evento o como condición de una búsqueda — porque su forma compacta se lee mejor en esos contextos.

```
// Ejemplo genérico (no pertenece al proyecto)
function duplicar(n) {           // declaración clásica
  return n * 2;
}
const duplicarFlecha = (n) => n * 2; // función flecha equivalente
```

**Fragmento real** — `src/db.js` y `src/routes/productos.js`. Las dos sintaxis conviven en el proyecto: la validación y el seed usan declaración; los manejadores de ruta son funciones flecha:

```
function poblarDesdeArchivo(rutaSql) {
```

```
router.get("/", (_req, res) => {
```

**Conexión con el proyecto:** cada ruta de la API es una función flecha que Express ejecuta cuando llega la petición correspondiente.

## 1.4 Objetos y arrays

**Teoría.** Un **objeto** agrupa datos relacionados bajo claves: `{ nombre: "Silla", precio: 10 }` tiene las claves `nombre` y `precio`, y a sus valores se accede con punto (`producto.nombre`). Un **array** es una lista ordenada de valores: `[a, b, c]`; se recorre con el bucle `for...of` y se puede ampliar con `push`. Los productos del proyecto son objetos con tres claves — `id`, `nombre`, `precio` — y la lista completa es un array de esos objetos: la combinación objeto + array es la forma universal de intercambiar colecciones de datos, dentro de la cual se inscribe el formato JSON (unidad 3.4).

```
// Ejemplo genérico (no pertenece al proyecto)
const item = { nombre: "Silla", precio: 10 }; // objeto
const lista = [item, { nombre: "Mesa", precio: 20 }]; // array de objetos
for (const uno of lista) {
  console.log(uno.nombre); // acceso con punto
}
```

**Fragmento real** — `public/index.html`. Al cargar la lista, cada producto se convierte en una fila; el array `valores` reúne lo que llevará cada celda:

```
const productos = await respuesta.json(); // JSON -> array de objetos
```

```
// toFixed(2): formato monetario, siempre 2 decimales
const valores = [producto.id, producto.nombre, `
${Number(producto.precio).toFixed(2)}`];
```

**Conexión con el proyecto:** la tabla del frontend es la representación visual de un array de objetos que llegó por HTTP.

## 1.5 Plantillas de texto (template literals)

**Teoría.** Concatenar textos con el operador `+` se vuelve frágil cuando hay que insertar valores. Las **plantillas de texto** (template literals) son textos delimitados por comillas invertidas (```) que admiten interpolar cualquier expresión con `${...}`: JavaScript evalúa la expresión y coloca su resultado dentro del texto. Son la forma estándar de armar URLs, mensajes y consultas legibles.

```
// Ejemplo genérico (no pertenece al proyecto)
const id = 7;
const url = `/api/items/${id}`; // resultado: "/api/items/7"
```

**Fragmento real** — `public/index.html` y `src/server.js`. Las URLs de edición y borrado, y el cartel de arranque, se construyen con esta sintaxis:

```
const url = creando ? '/api/productos' : `/api/productos/${idEnEdicion}`;
```

```
console.log(`Servidor corriendo en http://localhost:${PORT}`);
```

**Conexión con el proyecto:** sin template literals no existiría la ruta `PUT /api/productos/7`: el id debe incrustarse en la URL para identificar el producto a modificar.

## 1.6 Desestructuración

**Teoría.** La **desestructuración** extrae propiedades de un objeto — o elementos de un array — directamente en variables con nombre, en una sola instrucción: `const { a, b } = objeto` crea `a` y `b` con los valores de las claves homónimas. Es la abreviatura estándar de `const a = objeto.a;` `const b = objeto.b;`, y mejora la legibilidad cuando un módulo o un body entregan varios valores de una vez.

```
// Ejemplo genérico (no pertenece al proyecto)
const { nombre, precio } = { nombre: "Silla", precio: 10 };
// nombre === "Silla"; precio === 10
// En una ruta de API, el mismo patrón se aplicaría al body:
// const { nombre, precio } = req.body;
```

**Fragmento real** — `src/routes/productos.js` y `test/poblar.test.js`. El proyecto desestructura módulos enteros y filas de resultados:

```
const { db } = require("../db");
const { validarProducto } = require("../validators/productos");
```

```
const { c } = db.prepare("SELECT COUNT(*) AS c FROM productos").get();
```

**Conexión con el proyecto:** la conexión a la base llega a las rutas desestructurada del objeto que `src/db.js` exporta — `{ db, poblarDesdeArchivo }` — y las rutas toman solo lo que necesitan.

## 1.7 El método find()

**Teoría.** Los arrays ofrecen métodos de búsqueda de orden superior: reciben una función y aplican una regla a cada elemento. `find()` devuelve el **primer** elemento que cumple la condición — la función devuelve verdadero — o `undefined` si ninguno la cumple. Es la herramienta natural para localizar un elemento por su identificador sin recorrer la lista a mano.

```
// Ejemplo genérico (no pertenece al proyecto)
const items = [{ id: 1 }, { id: 2 }, { id: 3 }];
const buscado = items.find((i) => i.id === 2); // { id: 2 }
const inexistente = items.find((i) => i.id === 99); // undefined
```

**Fragmento real** — `public/index.html` . Al editar, el producto elegido se localiza en la lista ya cargada, sin un pedido extra al servidor:

```
function editarProducto(id) {
  // find() devuelve el primer elemento que cumple la condición
  // (o undefined si no está): se busca en la lista ya cargada.
  const producto = productosActuales.find((p) => p.id === id);
  if (!producto) return;
```

**Conexión con el proyecto:** `find()` convierte un clic sobre "Editar" en el objeto concreto que se carga en el formulario; el guard `if (!producto) return;` protege el caso de lista desactualizada.

## Unidad 2 — El navegador y el DOM (Nivel 2)

### 2.1 Referencias al DOM: `getElementById`

**Teoría.** El navegador representa la página como un árbol de objetos llamado **DOM** (Document Object Model): cada etiqueta HTML es un nodo que JavaScript puede leer y modificar. El atributo `id` identifica un nodo de forma única, y `document.getElementById("...")` devuelve ese nodo. La práctica recomendada — que el proyecto sigue — es buscar cada referencia una sola vez, guardarla en una constante y reutilizarla.

```
// Ejemplo genérico (no pertenece al proyecto)
// HTML: <button id="enviar">Enviar</button>
const boton = document.getElementById("enviar");
```

**Fragmento real** — `public/index.html` . Las cuatro referencias fijas de la página se resuelven al inicio del script:

```
// Referencias al DOM (la página). Se buscan una vez y se guardan
// en constantes: más legible y rápido que buscarlas en cada uso.
const formulario = document.getElementById('formulario-producto');
const tbody = document.getElementById('lista-productos');
const botonGuardar = document.getElementById('boton-guardar');
const botonCancelar = document.getElementById('boton-cancelar');
```

**Conexión con el proyecto:** toda la interfaz — formulario, tabla y botones — se maneja desde estas cuatro constantes; sin referencias al DOM no existe interfaz dinámica.

## 2.2 Eventos y addEventListener: submit, click y preventDefault

**Teoría.** Un **evento** es un suceso que el navegador anuncia: un clic, el envío de un formulario, una tecla. `elemento.addEventListener("evento", función)` registra la función — el **manejador** — que se ejecutará cuando el evento ocurra. El envío de formularios tiene un comportamiento por defecto molesto para las aplicaciones de una página: recargarla. `evento.preventDefault()` cancela ese comportamiento y deja el control en el JavaScript de la página.

```
// Ejemplo genérico (no pertenece al proyecto)
formulario.addEventListener("submit", (e) => {
  e.preventDefault(); // evita la recarga de la página
});
```

**Fragmento real** — `public/index.html`. El manejador del submit cancela la recarga antes de nada; el botón Cancelar reacciona a `click`:

```
formulario.addEventListener('submit', async (evento) => {
  // preventDefault() cancela el comportamiento por defecto del
  // submit (recargar la página). Sin esto, la recarga cortaría
  // nuestro JavaScript antes de llegar a enviar el pedido.
  evento.preventDefault();
```

```
botonCancelar.addEventListener('click', salirDeEdicion);
```

**Conexión con el proyecto:** sin `preventDefault()`, la recarga de página destruiría el flujo POST → GET que llena la tabla; con él, el submit se convierte en una petición HTTP controlada.

## 2.3 Creación de nodos: createElement, textContent y appendChild (frente a innerHTML)

**Teoría.** Existen dos maneras de agregar contenido desde JavaScript. La primera construye nodos: `document.createElement("td")` crea un elemento vacío, `nodo.textContent = valor` le inserta **texto plano** — sin interpretar HTML — y `padre.appendChild(hijo)` lo agrega al final del contenido del padre. La segunda, `elemento.innerHTML = "<td>...</td>"`, interpreta el texto como HTML y lo inserta analizado. La diferencia es una cuestión de seguridad: si el texto proviene de datos de usuarios, `innerHTML` ejecutaría cualquier etiqueta incluida — por ejemplo `<script>...` — y abriría la puerta al ataque **XSS** (*Cross-Site Scripting*: inyección de código que el navegador termina ejecutando). `textContent` inserta los mismos caracteres como texto visible, sin ejecutar nada. Vaciar un contenedor con `innerHTML = ''` sí es seguro, porque no interpola ningún dato.

```
// Ejemplo genérico (no pertenece al proyecto)
const fila = document.createElement("tr");
const celda = document.createElement("td");
```

```
celda.textContent = "<script>peligro</script>"; // se MUESTRA como texto
fila.appendChild(celda);
```

**Fragmento real** — `public/index.html` . La tabla se redibuja con la combinación segura, y el contenedor se vacía antes de cada listado:

```
// Vaciar y redibujar todo. Con muchos registros se optimizaría
// actualizando solo lo que cambió.
tbody.innerHTML = '';
```

```
celda.textContent = valor;
fila.appendChild(celda);
```

**Conexión con el proyecto:** un producto cuyo nombre contuviera HTML se mostraría como texto y jamás se ejecutaría, porque cada celda se llena con `textContent` .

## 2.4 Formularios: required, type number, parseFloat y reset

**Teoría.** Los formularios HTML incluyen ayudas de validación del lado del navegador: el atributo `required` impide enviar el formulario con el campo vacío, y `type="number"` (con `step` para los decimales permitidos) restringe el contenido. Dos detalles de JavaScript completan el trabajo. Primero: `.value` devuelve **siempre texto**, así que un precio debe convertirse con `parseFloat()` antes de tratarse como número. Segundo: `formulario.reset()` limpia todos los campos de una vez, y sirve para volver el formulario a su estado inicial tras cada operación.

```
// Ejemplo genérico (no pertenece al proyecto)
// HTML: <input id="precio" type="number" required>
const texto = document.getElementById("precio").value; // siempre string
const numero = parseFloat(texto);                      // número real
formulario.reset();                                     // limpia los campos
```

**Fragmento real** — `public/index.html` . El formulario declara sus validaciones visuales; el submit convierte el precio; la salida de edición limpia con `reset()` :

```
<form id="formulario-producto">
  <!-- required: el navegador valida solo que no esté vacío -->
  <input type="text" id="nombre" placeholder="Nombre del producto" required>
  <!-- step 0.01: admite decimales (precios con centavos) -->
  <input type="number" step="0.01" id="precio" placeholder="Precio" required>
```

```
// .value siempre devuelve texto; para el precio hace falta
// parseFloat() y así trabajar con un número de verdad.
```

```
const nombre = document.getElementById('nombre').value;
const precio = parseFloat(document.getElementById('precio').value);
```

```
function salirDeEdicion() {
  idEnEdicion = null;
  formulario.reset();
  botonGuardar.textContent = 'Guardar';
  botonCancelar.hidden = true;
}
```

**Conexión con el proyecto:** el `required` y el `type number` ayudan a la persona usuaria, pero — como se verá en la unidad 6.1 — el servidor repite y supera esas verificaciones, porque el navegador puede saltarse.

## 2.5 Clausuras: un botón por fila

**Teoría.** Una **clausura** (closure) es una función que recuerda las variables del entorno donde fue creada, aunque ese entorno ya haya terminado de ejecutarse. El caso de uso clásico son los elementos creados dentro de un bucle: cada función flecha creada en una iteración captura el valor de esa iteración, y cada botón guarda "su" dato particular. Sin clausuras, todos los botones compartirían la última variable del bucle.

```
// Ejemplo genérico (no pertenece al proyecto)
for (const item of items) {
  boton.addEventListener("click", () => usar(item.id)); // recuerda ESE item.id
}
```

**Fragmento real** — `public/index.html` . Cada fila recibe su botón Borrar, cuya función flecha captura el id de la fila que la creó:

```
const botonBorrar = document.createElement('button');
botonBorrar.textContent = 'Borrar';
// La arrow function hace de "clausura": guarda el producto.id de
// ESTA fila y solo se ejecuta cuando se hace clic en ese botón.
botonBorrar.addEventListener('click', () => borrarProducto(producto.id));
celdaAccion.appendChild(botonBorrar);
fila.appendChild(celdaAccion);
tbody.appendChild(fila);
```

**Conexión con el proyecto:** la clausura es lo que hace que cada botón Borrar sepa exactamente qué producto borrar, aunque todos los botones fueran creados por el mismo bucle.

## Unidad 3 — Asincronismo y HTTP (Nivel 3)

### 3.1 El modelo petición/respuesta

**Teoría.** HTTP (HyperText Transfer Protocol) es el protocolo — el conjunto de reglas — con el que se comunican los navegadores y los servidores web. El modelo es estrictamente por turnos: el **cliente** envía una **petición** (request) que indica qué quiere — un verbo y una ruta — y el **servidor** devuelve una **respuesta** (response) con un código de estado y, en general, un cuerpo de datos. El frontend y el backend de este proyecto son dos programas que solo se conocen a través de ese diálogo: no comparten código ni memoria, solo mensajes HTTP.

```
// Ejemplo genérico (no pertenece al proyecto)
petición:  GET /items
respuesta: 200 OK – cuerpo: '[{"id":1,"nombre":"Silla"}]'
```

**Fragmento real** — `public/index.html` . El cliente pide la lista y parsea la respuesta; el servidor equivalente responde en dos líneas:

```
const respuesta = await fetch('/api/productos'); // GET es el default
const productos = await respuesta.json(); // JSON -> array de objetos
```

**Conexión con el proyecto:** cada acción de la página — listar, crear, editar, borrar — es exactamente un par petición/respuesta; el mapa completo del diálogo está en el encabezado de `src/app.js` .

### 3.2 Los verbos GET, POST, PUT y DELETE

**Teoría.** Cada petición HTTP lleva un **verbo** que declara la intención. `GET` obtiene datos sin modificar nada; `POST` crea un recurso y lleva los datos en el cuerpo; `PUT` reemplaza un recurso existente identificado por su URL; `DELETE` lo elimina. Combinar verbos con rutas que nombran recursos — `/api/productos` , `/api/productos/7` — es la esencia del estilo **REST**, la convención de diseño de APIs que este proyecto sigue.

```
// Ejemplo genérico (no pertenece al proyecto)
GET    /items    → obtener la lista
POST   /items    → crear un elemento
PUT    /items/7  → reemplazar el elemento 7
DELETE /items/7  → borrar el elemento 7
```

**Fragmento real** — `public/index.html` . El mismo formulario envía POST o PUT según el modo; el id viaja en la URL al editar:

```
// ¿Creando o editando? Misma ruta base; cambia el método y, al
// editar, el id viaja en la URL: PUT /api/productos/7.
const creando = idEnEdicion === null;
const url = creando ? '/api/productos' : `/api/productos/${idEnEdicion}`;
```

```
const respuesta = await fetch(url, {
  method: creando ? 'POST' : 'PUT',
```

**Conexión con el proyecto:** las cuatro rutas de `src/routes/productos.js` — `router.get`, `router.post`, `router.put`, `router.delete` — son la contraparte servida de estos verbos.

### 3.3 Los códigos de estado

**Teoría.** Toda respuesta HTTP incluye un **código de estado** numérico que resume el resultado. La familia 2xx comunica éxito: **200** OK (con contenido), **201** Created (se creó un recurso), **204** No Content (éxito sin nada que devolver). La familia 4xx comunica un error del cliente: **400** Bad Request (la petición es inválida) y **404** Not Found (el recurso no existe). Un detalle importante para el frontend: `fetch` no lanza excepción ante un 4xx; la propiedad `respuesta.ok` (verdadera solo para la familia 2xx) debe revisarse a mano.

```
// Ejemplo genérico (no pertenece al proyecto)
200 → éxito con contenido      400 → datos inválidos
201 → creado                   404 → no existe
204 → éxito sin contenido
```

**Fragmento real** — `src/routes/productos.js`. La ruta DELETE ejemplifica las tres situaciones: éxito sin contenido (204) y el par éxito/error según exista el id:

```
router.delete("/:id", (req, res) => {
  const borrar = db.prepare("DELETE FROM productos WHERE id = ?");
  // run() devuelve "changes": cuántas filas fueron borradas de verdad.
  const resultado = borrar.run(req.params.id);
  // Si changes es 0, no existía ningún producto con ese id. Responder
  // 404 (Not Found) es más honesto que un 204 silencioso: quien llamó
  // pidió borrar algo que no estaba.
  if (resultado.changes === 0) {
    return res.status(404).json({ error: "Producto no encontrado" });
  }
  // 204 No Content: "salió bien y no tengo nada que devolver". Es la
  // convención REST para borrados exitosos. .end() cierra la respuesta
  // sin body.
  res.status(204).end();
});
```

**Conexión con el proyecto:** el frontend consulta `respuesta.ok` en cada guardado y borrado; su decisión de limpiar o no el formulario depende directamente del código recibido.

### 3.4 JSON: stringify y parse

**Teoría.** **JSON** (JavaScript Object Notation) es un formato de texto para intercambiar datos, cuya sintaxis copia la de objetos y arrays de JavaScript. Como `fetch` solo envía texto y las respuestas llegan como texto, hacen falta dos conversiones: **serializar** (objeto → texto) con `JSON.stringify(objeto)` y

**parsear** (texto → objeto) con `JSON.parse(texto)` . Para las respuestas, `respuesta.json()` hace el parseo y devuelve una promesa con el objeto ya formado.

```
// Ejemplo genérico (no pertenece al proyecto)
const texto = JSON.stringify({ nombre: "Silla" }); // '{"nombre":"Silla"}'
const objeto = JSON.parse(texto);                // { nombre: "Silla" }
```

**Fragmento real** — `public/index.html` . El body de la petición se serializa; la respuesta del guardado se parsea para leer el id generado:

```
// fetch solo envía texto: el objeto debe serializarse
body: JSON.stringify({ nombre, precio })
```

```
const datos = await respuesta.json(); // { id, nombre, precio }
```

**Conexión con el proyecto:** JSON es el idioma común de las dos mitades del proyecto: el servidor responde con `res.json(...)` y el navegador envía y lee con `JSON.stringify` y `respuesta.json()` .

### 3.5 async/await: la secuencia guardar y después refrescar

**Teoría.** Las operaciones de red tardan: mientras una petición viaja, el resto de la página debe seguir viva. JavaScript resuelve esto con la **asincronía**: una función marcada `async` puede usar `await` delante de una operación lenta — representada por una **promesa**, el valor "pendiente" — y pausarse ahí sin congelar la página; cuando el resultado llega, la función continúa desde ese punto. El valor pedagógico clave es que `await` **secuencia**: la línea siguiente no se ejecuta hasta que la línea con `await` termina.

```
// Ejemplo genérico (no pertenece al proyecto)
async function guardarYRefrescar() {
  await guardar(); // primero termina el guardado...
  await refrescar(); // ...recién después se refresca
}
```

**Fragmento real** — `public/index.html` . El submit espera el resultado del guardado y solo si fue exitoso limpia el formulario y pide la lista actualizada:

```
// await: espera el resultado del guardado (POST o PUT, según
// el modo) antes de continuar. Si falla, guardado es false y
// el formulario NO se limpia: no se pierde lo escrito por un
// error del servidor.
const guardado = await guardarProducto(nombre, precio);
if (guardado) {
  // salirDeEdicion() limpia el formulario Y devuelve el estado a
```

```
// "creando" (texto del botón incluido). Sirve para ambos caminos.
salirDeEdicion();
// Ciclo POST -> GET: recién ahora que el producto está en la
// base, se pide la lista actualizada. El await garantiza ese
// orden (sin él, ambos correrían en paralelo).
await cargarProductos();
}
```

**Conexión con el proyecto:** sin el `await`, listar y guardar correrían en paralelo y la tabla podría refrescarse antes de que el producto existiera; el `await` garantiza el orden correcto del ciclo POST → GET.

### 3.6 fetch: method, headers, body y rutas relativas

**Teoría.** `fetch(url, opciones)` es la función del navegador que envía peticiones HTTP y devuelve una promesa con la respuesta. El segundo argumento, opcional para GET, precisa el verbo ( `method` ), los **encabezados** ( `headers` : metadatos de la petición, como el tipo de contenido del body) y el cuerpo ( `body` : los datos). Las **rutas relativas** — `"/api/productos"`, sin dominio — se completan con el origen de la página actual: si la página vino de `http://localhost:3000`, la petición va a `http://localhost:3000/api/productos`. Por eso el HTML debe servirse por HTTP y no abrirse como archivo `file://`: sin origen, la ruta relativa no tiene contra qué resolverse.

```
// Ejemplo genérico (no pertenece al proyecto)
const r = await fetch("/api/items", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(dato),
});
```

**Fragmento real** — `public/index.html`. La petición de guardado completa: método condicional, encabezado de tipo JSON y body serializado:

```
const creando = idEnEdicion === null;
const url = creando ? '/api/productos' : `/api/productos/${idEnEdicion}`;
const respuesta = await fetch(url, {
  method: creando ? 'POST' : 'PUT',
  // El backend necesita saber que el body es JSON
  headers: { 'Content-Type': 'application/json' },
  // fetch solo envía texto: el objeto debe serializarse
  body: JSON.stringify({ nombre, precio })
});
```

**Conexión con el proyecto:** el encabezado `Content-Type: application/json` es lo que permite al servidor interpretar el body; sin él ni `express.json()` ni las rutas podrían leer el producto.

## Unidad 4 — El servidor con Node.js y Express (Nivel 4)

### 4.1 Node.js y npm

**Teoría.** **Node.js** es un entorno que ejecuta JavaScript fuera del navegador — sobre todo en servidores — y le agrega las capacidades que una página no tiene: abrir archivos, escuchar puertos de red. **npm** es su gestor de paquetes: lee el archivo `package.json`, que declara las librerías de terceros que el proyecto necesita (sus **dependencias**), las descarga en `node_modules/` con `npm install` y ejecuta los comandos definidos en la sección `scripts`. Este proyecto requiere Node 22 o superior, condición que impone la librería `better-sqlite3`.

```
// Ejemplo genérico (no pertenece al proyecto)
npm install          → descarga las dependencias declaradas
npm test            → ejecuta el script "test" de package.json
node src/servidor.js → ejecuta un archivo con Node
```

**Fragmento real** — `package.json`. Los dos scripts del proyecto y sus dependencias:

```
"scripts": {
  "test": "node --test \"test/**/*.test.js\"",
  "start": "node src/server.js"
},
```

**Conexión con el proyecto:** `npm test` — las 13 pruebas — y `npm start` — el servidor — son los dos comandos diarios, y ambos salen de este archivo.

### 4.2 Módulos CommonJS: `require` y `module.exports`

**Teoría.** Node organiza el código en **módulos**: cada archivo es un módulo con su ámbito propio. El sistema CommonJS define dos instrucciones: `module.exports` declara qué exporta el archivo — un valor cualquiera, habitualmente un objeto — y `require("./ruta")` lo importa desde otro archivo. Un detalle con consecuencias útiles: Node guarda en **caché** los módulos ya cargados, de modo que dos `require` del mismo archivo reciben exactamente el mismo objeto. Ese es el mecanismo que permite a las pruebas cerrar la misma conexión que abrió la app.

```
// Ejemplo genérico (no pertenece al proyecto)
// utilidades.js
module.exports = { sumar };
// principal.js
const { sumar } = require("./utilidades");
```

**Fragmento real** — `src/app.js` y `src/db.js`. Importaciones al inicio de un módulo; exportación al final:

```
const path = require("path"); // Utilidades para rutas de archivos del sistema operativo
const express = require("express");
const productosRouter = require("../routes/productos");
```

```
module.exports = { db, poblarDesdeArchivo };
```

**Conexión con el proyecto:** toda la arquitectura modular de `src/` se sostiene en este par de instrucciones: cada archivo exporta una sola cosa bien definida — la app, la conexión, el router, el validador — y quien lo requiere decide qué usar.

## 4.3 La aplicación y su cadena de middlewares

**Teoría.** `express()` crea el objeto **aplicación**, el corazón de un servidor Express. Un **middleware** es una función por la que Express hace pasar cada petición; los middlewares y las rutas se ejecutan **en el orden en que se registran**, de arriba hacia abajo, hasta que alguno responde y corta la cadena. Pensar el servidor como una cadena ordenada — primero interpretar el body, después servir archivos, después las rutas — es la idea central del framework.

```
// Ejemplo genérico (no pertenece al proyecto)
const app = express();
app.use(middlewareA); // se ejecuta primero
app.use(middlewareB); // se ejecuta después
```

**Fragmento real** — `src/app.js` . La creación de la app y su cadena, con el orden comentado en el propio archivo:

```
// Una "app" de Express procesa cada pedido HTTP pasando por una
// cadena de middlewares: funciones que se ejecutan en orden, de
// arriba hacia abajo, hasta que algo responda.
const app = express();
```

**Conexión con el proyecto:** el orden de `src/app.js` no es decorativo: `express.json()` debe preceder a las rutas, o `req.body` llegaría vacío a los manejadores que lo leen.

## 4.4 `express.json`

**Teoría.** `express.json()` es un middleware incluido en Express que interpreta el cuerpo JSON de las peticiones POST y PUT: lee el texto, lo convierte en objeto JavaScript y lo deja en `req.body` . Es el complemento del lado del servidor del encabezado `Content-Type: application/json` que el frontend envía (unidad 3.6): uno declara el formato y el otro lo interpreta. Sin este middleware, `req.body` vale `undefined` y toda validación fallaría.

```
// Ejemplo genérico (no pertenece al proyecto)
app.use(express.json()); // habilita req.body en POST/PUT con JSON
```

**Fragmento real** — `src/app.js` . Primer middleware registrado:

```
// Middleware: interpreta el body JSON de los pedidos POST/PUT y lo
// deja disponible en req.body como objeto JavaScript.
// Sin esta línea, req.body llegaría undefined.
app.use(express.json());
```

**Conexión con el proyecto:** gracias a esta línea, `validarProducto(req.body)` en las rutas POST y PUT recibe un objeto real con `nombre` y `precio` .

## 4.5 express.static

**Teoría.** `express.static(carpeta)` es el middleware que sirve archivos estáticos — HTML, CSS, imágenes — tal como están en el disco: si el navegador pide `/` , Express entrega el `index.html` de esa carpeta; si pide `/estilos.css` , entrega ese archivo. Elegir la carpeta es una decisión de seguridad: solo queda expuesto lo que esté dentro de ella.

```
// Ejemplo genérico (no pertenece al proyecto)
app.use(express.static("./sitio")); // sirve ./sitio/index.html en "/"
```

**Fragmento real** — `src/app.js` . El frontend se sirve desde `public/` , un nivel arriba de `src/` :

```
// __dirname = ruta absoluta de la carpeta donde está este archivo.
// Como ahora este archivo vive en src/, se sube un nivel con ".."
// para llegar a la carpeta public/ de la raíz del proyecto.
app.use(express.static(path.join(__dirname, "..", "public")));
```

**Conexión con el proyecto:** esta única línea convierte `public/index.html` en la página de `http://localhost:3000/` y, al mismo tiempo, deja fuera del alcance HTTP la base de datos y el código del servidor.

## 4.6 Parámetros de ruta: req.params.id

**Teoría.** Una ruta puede tener segmentos variables: escribir `/:id` en la definición declara "aquí va un valor que cambia por pedido". Express extrae ese valor de la URL y lo entrega en `req.params.id` . Dos precisiones importantes: el nombre tras los dos puntos es el de la propiedad; y el valor llega siempre como **string**, aunque represente un número, por lo que a veces hay que convertirlo con `Number()` .

```
// Ejemplo genérico (no pertenece al proyecto)
app.put("/items/:id", (req, res) => {
```

```
const id = req.params.id; // "7" si la URL fue /items/7
});
```

**Fragmento real** — `src/routes/productos.js`. La ruta DELETE declara el parámetro y lo usa; el comentario del propio archivo advierte sobre el tipo:

```
// DELETE /api/productos/:id → borra el producto con ese id.
// ":id" es un PARÁMETRO DE RUTA: viaja en la URL (no en el body) y
// Express lo deja disponible en req.params.id. Ojo: llega como STRING
// ("7"), pero SQLite lo compara bien contra el id numérico.
router.delete("/:id", (req, res) => {
```

```
res.json({ id: Number(req.params.id), nombre, precio });
```

**Conexión con el proyecto:** el id que la clausura del botón Borrar incrustó en la URL (unidad 2.5) reaparece del lado del servidor como `req.params.id`: es el mismo dato cruzando la frontera HTTP.

## 4.7 Router: agrupar y montar rutas

**Teoría.** `express.Router()` crea una **mini-aplicación** que agrupa rutas relacionadas con sus propios métodos HTTP. La aplicación principal la "monta" con `app.use(prefijo, router)`: todas las rutas internas del router se sirven bajo ese prefijo. El resultado es una organización escalable: el router de productos conoce sus rutas como `/` y `/:id`, y solo la app sabe que viven bajo `/api/productos`.

```
// Ejemplo genérico (no pertenece al proyecto)
const rutas = express.Router();
rutas.get("/", listar);
app.use("/api/items", rutas); // "/" interno → /api/items
```

**Fragmento real** — `src/routes/productos.js` y `src/app.js`. Creación interna y montaje externo:

```
const router = express.Router();
```

```
// Se monta el router de productos bajo el prefijo /api/productos:
// las rutas que dentro de routes/productos.js se definen como "/"
// y "/:id" se sirven en /api/productos y /api/productos/:id.
app.use("/api/productos", productosRouter);
```

**Conexión con el proyecto:** el router es la pieza que permite que `routes/productos.js` hable de `/` y `/:id` sin saber dónde se lo va a colgar — una separación limpia entre definición y montaje.

## Unidad 5 — Persistencia de datos (Nivel 5)

### 5.1 SQLite: una base embebida en un archivo

**Teoría.** Una **base de datos** es un sistema que almacena datos organizados en **tablas** — grillas de filas y columnas — y los consulta con un lenguaje específico. Las bases más conocidas funcionan como servidores separados que hay que instalar y conectar. **SQLite** pertenece a la otra familia, las bases **embebidas**: el motor vive dentro del propio programa y toda la base — esquema y datos — cabe en un único archivo del disco. Para un proyecto de aprendizaje esto significa cero instalación: la base nace la primera vez que el programa abre el archivo.

```
// Ejemplo genérico (no pertenece al proyecto)
aplicacion.js —abre—▶ datos.db  (todo el contenido en un solo archivo)
```

**Fragmento real** — `src/db.js` . La apertura crea el archivo si falta:

```
// Abre la base (o crea el archivo si no existe). No hay un servidor
// de base de datos separado: todo queda en un único archivo .db en
// el disco (mi_base_de_datos.db, salvo que DB_PATH diga otra cosa).
const db = new Database(DB_PATH);
```

**Conexión con el proyecto:** `mi_base_de_datos.db` — el archivo que aparece en la raíz del proyecto — es la base completa; borrarlo equivale a empezar de cero.

### 5.2 better-sqlite3: un driver síncrono

**Teoría.** Un **driver** es la librería que conecta el programa con la base de datos y traduce sus llamadas al motor. `better-sqlite3` es el driver de este proyecto y su rasgo distintivo es ser **síncrono**: cada consulta se ejecuta y devuelve su resultado de inmediato, sin promesas ni callbacks — no hace falta `await` . Esto simplifica el código de las rutas (el resultado está en la línea siguiente) a cambio de bloquear el proceso durante la consulta, comportamiento aceptable para consultas breves como las de este proyecto.

```
// Ejemplo genérico (no pertenece al proyecto)
const filas = consulta.all(); // resultado inmediato, sin await
```

**Fragmento real** — `src/db.js` y `src/routes/productos.js` . La naturaleza síncrona, declarada en el encabezado, y su efecto visible: ninguna ruta usa `await` para consultar:

```
// better-sqlite3: driver de SQLite. Es SINCRÓNICO: cada consulta
// bloquea hasta terminar (no usa callbacks ni promesas).
// La base completa vive en un solo archivo .db en el disco.
```

```
const productos = db.prepare("SELECT * FROM productos").all();
```

**Conexión con el proyecto:** la asincronía del proyecto está toda del lado del navegador (los `fetch` con `await`); del lado del servidor, las consultas SQLite son llamadas directas que devuelven el resultado al instante.

## 5.3 SQL básico: CREATE, SELECT, INSERT, UPDATE y DELETE

**Teoría. SQL** (Structured Query Language) es el lenguaje de las bases relacionales. Cinco sentencias cubren el proyecto. `CREATE TABLE IF NOT EXISTS` define la tabla una sola vez, sin fallar si ya existe. `SELECT` lee filas. `INSERT` agrega filas. `UPDATE` modifica — y requiere `WHERE` para limitar qué filas. `DELETE` elimina — con la misma advertencia: sin `WHERE`, afecta **todas** las filas. El `WHERE` ("donde") es la cláusula que selecciona las filas involucradas.

```
-- Ejemplo genérico (no pertenece al proyecto)
CREATE TABLE IF NOT EXISTS tareas (id INTEGER PRIMARY KEY, titulo TEXT);
INSERT INTO tareas (titulo) VALUES ('Estudiar');
SELECT * FROM tareas;
UPDATE tareas SET titulo = 'Estudiar más' WHERE id = 1;
DELETE FROM tareas WHERE id = 1;
```

**Fragmento real** — `src/db.js` y `poblar_base.sql`. La tabla del proyecto y los datos de ejemplo:

```
db.exec(`
  CREATE TABLE IF NOT EXISTS productos (
    id INTEGER PRIMARY KEY AUTOINCREMENT, -- id autogenerado: 1, 2, 3...
    nombre TEXT NOT NULL,                -- texto obligatorio
    precio REAL NOT NULL                  -- número con decimales
  )
`);
```

```
INSERT INTO productos (nombre, precio) VALUES
('Teclado Mecánico', 45.99),
('Mouse Inalámbrico', 25.50),
('Monitor 24 Pulgadas', 179.00),
('Auriculares Bluetooth', 59.90),
('Pad Mouse Gamer', 12.00);
```

**Conexión con el proyecto:** las cinco sentencias tienen su lugar exacto — `CREATE TABLE` en `src/db.js`, `INSERT` en `poblar_base.sql`, y `SELECT`, `UPDATE` y `DELETE` en las rutas — de modo que el CRUD completo se lee como SQL mínimo y real.

## 5.4 Prepared statements: la defensa contra la inyección SQL

**Teoría.** Un **prepared statement** (consulta preparada) es una consulta que se compila con marcadores `?` en lugar de valores; los valores se pasan después, por separado, en el `run(...)`. El motor separa estructuralmente el SQL de los datos: por más que un dato contenga caracteres especiales, nunca se interpreta como SQL. Eso cierra la **inyección SQL**, el ataque clásico que consiste en introducir SQL malicioso a través de un campo de datos — por ejemplo, un nombre que contenga comillas y una sentencia extra. La regla resultante es absoluta: jamás se concatenan valores de usuario dentro del texto SQL.

```
// Ejemplo genérico (no pertenece al proyecto)
// INSEGURO: "SELECT * FROM t WHERE nombre = '" + nombre + "'"
db.prepare("SELECT * FROM t WHERE nombre = ?").get(nombre); // seguro
```

**Fragmento real** — `src/routes/productos.js`. El INSERT del proyecto: dos marcadores, dos valores por separado; el resultado trae el id autogenerado:

```
// Ya validado: trim() normaliza el nombre (sin espacios sobrantes).
const nombre = req.body.nombre.trim();
const precio = req.body.precio;
const insert = db.prepare(
  "INSERT INTO productos (nombre, precio) VALUES (?, ?)",
);
// run() ejecuta el INSERT; lastInsertRowid trae el id generado.
const resultado = insert.run(nombre, precio);
// 201 Created: convención REST para "se creó un recurso nuevo".
res.status(201).json({ id: resultado.lastInsertRowid, nombre, precio });
```

También la prueba del seed usa un marcador para buscar por nombre:

```
const teclado = db
  .prepare("SELECT nombre, precio FROM productos WHERE nombre = ?")
  .get("Teclado Mecánico");
```

**Conexión con el proyecto:** las cuatro consultas del CRUD usan `?`; un producto llamado `'`); `DELETE FROM productos; --` quedaría guardado como texto inofensivo en lugar de vaciar la tabla.

## Unidad 6 — Integración y calidad (Nivel 6)

### 6.1 Validación del lado del servidor (y la trampa de `!valor` frente a `Number.isFinite`)

**Teoría.** Validar en el **cliente** — con `required` y `type number` — mejora la experiencia, pero es saltable: herramientas como `curl` envían peticiones sin pasar por ningún formulario. La validación del **servidor** es la única obligatoria, porque es la última barrera antes de los datos. Dos principios la

distinguen en este proyecto. Primero, se valida **antes** de tocar la base. Segundo, se **acumulan todos los errores** en un array y se devuelven juntos, para que quien consume la API corrija todo de una vez. Además, el proyecto documenta una trampa clásica: para exigir un número no sirve **!precio** (negación), porque rechazaría sin distinguir cualquier valor *falsey* — entre ellos valores legítimos en otros contextos —; **Number.isFinite** acepta exactamente los números finitos y rechaza **NaN**, textos como **"10"**, **Infinity** y objetos.

```
// Ejemplo genérico (no pertenece al proyecto)
const errores = [];
if (typeof dato.nombre !== "string") errores.push("nombre inválido");
return errores; // array vacío = válido
```

**Fragmento real** — `src/validators/productos.js`. La regla del precio, con la trampa advertida en el propio código:

```
// --- precio ---
// Number.isFinite acepta 10 y 10.5; rechaza NaN, "10" (string),
// Infinity y objetos. TRAMPA CLÁSICA: no usar !precio, porque
// además de NaN rechazaría cualquier valor falsy sin distinguir.
if (!Number.isFinite(datos.precio)) {
  errores.push("El precio es obligatorio y debe ser un número.");
} else if (datos.precio < 0.01) {
  // Regla de negocio: nada gratis, el mínimo es 0.01.
  errores.push("El precio mínimo permitido es 0.01.");
}

return errores;
}
```

**Conexión con el proyecto:** el guard inicial `const datos = body || {}` completa la defensa — una petición sin body valida como objeto vacío y responde 400 ordenada, sin romper el servidor.

## 6.2 Semántica de los códigos: 201, 204, 400, 404 y el patrón `changes === 0`

**Teoría.** Responder "con el código correcto" es una forma de comunicar intención. La convención REST que el proyecto aplica: **201** cuando se creó un recurso (la respuesta muestra el recurso con su id); **204** cuando el éxito no deja contenido que devolver; **400** cuando los datos no cumplen las reglas (con el detalle de los errores); **404** cuando el recurso pedido no existe. El caso más interesante es el 404 por `changes === 0`: las sentencias UPDATE y DELETE reportan cuántas filas afectaron; cero filas significa que el id no existía, y responder 404 es más honesto que un éxito silencioso que no hizo nada.

```
// Ejemplo genérico (no pertenece al proyecto)
crear exitoso      → 201      crear inválido    → 400
editar exitoso     → 200      id inexistente → 404
borrar exitoso     → 204      id inexistente → 404
```

**Fragmento real** — `src/routes/productos.js` . La creación responde 201 con el producto; la edición traduce `changes === 0` en 404:

```
// 201 Created: convención REST para "se creó un recurso nuevo".
res.status(201).json({ id: resultado.lastInsertRowid, nombre, precio });
```

```
// Mismo patrón changes que en DELETE: 0 significa "ese id no existe".
if (resultado.changes === 0) {
  return res.status(404).json({ error: "Producto no encontrado" });
}
```

**Conexión con el proyecto:** las pruebas automatizadas fijan esta semántica como contrato — hay casos específicos para 201, 204, 400 y 404, incluidos PUT y DELETE sobre ids inexistentes.

## 6.3 Variables de entorno: DB\_PATH y PORT

**Teoría.** Una **variable de entorno** es un valor que el sistema operativo entrega al proceso al arrancarlo, sin que figure en el código. En Node se leen con `process.env.NOMBRE` , y el patrón `process.env.X || "valor por defecto"` combina flexibilidad con un comportamiento sensato cuando la variable no viene. Sirven para lo que cambia por entorno — una ruta de archivo, un puerto — sin editar el programa. El proyecto las usa en dos lugares: la ruta de la base y el puerto de escucha.

```
# Ejemplo genérico (no pertenece al proyecto)
DB_PATH=/tmp/prueba.db PORT=3100 node servidor.js
```

**Fragmento real** — `src/db.js` y `src/server.js` . El mismo criterio, variable con valor por defecto:

```
// La ruta del archivo de base se puede cambiar con la variable de
// entorno DB_PATH (los tests la apuntan a un archivo temporal para
// no tocar la base real). Sin esa variable, se usa la de siempre.
const DB_PATH = process.env.DB_PATH || "mi_base_de_datos.db";
```

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor corriendo en http://localhost:${PORT}`);
});
```

**Conexión con el proyecto:** `DB_PATH` es, además, la bisagra del aislamiento de las pruebas: todo el sistema de bases temporales de la unidad 6.5 depende de esta única línea.

## 6.4 Estructura modular del proyecto

**Teoría.** Organizar un proyecto en **módulos** consiste en repartir responsabilidades entre archivos con un propósito único cada uno, conectados por interfaces claras — aquí, `require` y `module.exports`. Los beneficios son concretos: cada archivo se entiende sin leer los demás, cada responsabilidad se prueba por separado y un cambio en una capa no arrastran a las otras. La estructura de `src/` del proyecto es un ejemplo canónico de esta separación en tamaño real.

```
// Ejemplo genérico (no pertenece al proyecto)
app.js (armar la app) · server.js (arrancar) · db.js (datos)
routes/ (rutas) · validators/ (reglas) · public/ (interfaz)
```

**Fragmento real** — `src/app.js`. El mapa de la API que encabeza el archivo funciona a la vez como documentación y como índice del sistema modular:

```
// Mapa de la API (todas las rutas bajo /api/productos):
// GET    /api/productos      → lista todos           (200)
// POST   /api/productos      → crea uno; valida los datos   (201)
// PUT    /api/productos/:id  → reemplaza uno; valida datos (200)
// DELETE /api/productos/:id  → borra uno                   (204)
// Errores posibles: 400 (datos inválidos) · 404 (id inexistente)
```

**Conexión con el proyecto:** la separación `app / arranque / datos / rutas / validación / interfaz` es lo que permite que las pruebas requieran la app sin arrancarla y que el frontend jamás toque la base.

## 6.5 Pruebas automatizadas: `node:test`, `supertest` y aislamiento con base temporal

**Teoría.** Una **prueba automatizada** es un pequeño programa que ejercita el código y verifica resultados esperados sin intervención humana: si alguien rompe un comportamiento, las pruebas lo delatan.

`node:test` es el corredor de pruebas que Node incluye de fábrica (función `test`, ganchos como `after`); `node:assert` aporta las verificaciones (`assert.strictEqual`, `assert.deepStrictEqual`); `supertest` dispara peticiones HTTP contra la app Express **sin levantar un servidor en un puerto real**. La pieza de ingeniería decisiva es el **aislamiento**: como `src/db.js` abre la base apenas se lo requiere y su ruta sale de `DB_PATH`, las pruebas definen esa variable — apuntando a un archivo temporal único — **antes** de cualquier `require` de `src/`. Así corren contra una base desechable y jamás tocan `mi_base_de_datos.db`.

```
// Ejemplo genérico (no pertenece al proyecto)
test("suma dos números", () => {
  assert.strictEqual(sumar(2, 3), 5);
});
```

**Fragmento real** — `test/api.test.js`. El orden crítico (variable primero, requires después) y la limpieza final:

```
// Va PRIMERO, antes de requerir la app: DB_PATH apunta a un archivo
// temporal con nombre único (Date.now()) evita colisiones entre
// corridas del mismo test o de dos terminales a la vez).
process.env.DB_PATH = path.join(os.tmpdir(), `zz-js01-test-${Date.now()}.db`);
```

```
after(() => {
  db.close();
  fs.rmSync(process.env.DB_PATH, { force: true });
});
```

**Conexión con el proyecto:** las 13 pruebas del proyecto — 12 de API en `test/api.test.js` y 1 de seed en `test/poblar.test.js` — corren con `npm test` contra bases temporales, y cada archivo de prueba usa el suyo, propio e irrepetible.

---

## Síntesis final

---

Los seis niveles de este material reconstruyen, pieza por pieza, la resolución completa del proyecto `zz-js01`. Los fundamentos del lenguaje (Nivel 1) escriben cada línea: variables y funciones declaran, objetos y arrays transportan los productos, los template literals arman las URLs y la desestructuración reparte los valores. El navegador y su DOM (Nivel 2) convierten esos datos en interfaz: referencias, eventos, nodos creados con `textContent` y clausuras que dan a cada botón su propio producto. El asincronismo y HTTP (Nivel 3) tienden el puente: peticiones con verbos y códigos de estado, datos en JSON, `await` que garantiza guardar antes de refrescar. Node y Express (Nivel 4) reciben ese puente del otro lado: módulos que exportan una sola responsabilidad, una cadena de middlewares que interpreta el body y sirve la interfaz, parámetros de ruta y un router que ordenan la API. La persistencia (Nivel 5) fija los resultados en disco: una base embebida en un archivo, manejada por un driver síncrono con las cinco sentencias SQL y protegida por prepared statements. Y la integración y calidad (Nivel 6) cierra el círculo: validación que no confía en nadie, códigos elegidos con precisión, configuración por variables de entorno, estructura modular y trece pruebas que verifican todo lo anterior contra bases temporales. Estudiado en este orden, cada concepto del proyecto deja de ser un tema aislado y pasa a ser una pieza identificable de una aplicación que funciona: la misma página que muestra la tabla, el mismo id que viaja de la clausura del botón a la URL, al parámetro de ruta, al `WHERE id = ?` y de vuelta al 404 honesto. Ese recorrido completo — del clic al disco y del disco a la tabla — es la resolución del proyecto, y es también la demostración de que cada concepto aquí desarrollado tiene un lugar concreto, verificable y necesario en el código.